

Switching a fleet to ONE shared `shapefile_config.json`

Requires MaterialClassification 1.1.5 or newer. On 1.1.4 and earlier the variable below is simply ignored — every box reads its own local file, always.

Normally each machine keeps its own `shapefile_config.json` (SDE connection, layer names, road-width fields, water mask), so changing a layer name means visiting every computer. Set one environment variable and a box reads a **shared** file instead — edit that single file and every box pointing at it follows on its next run.

JARVIS cannot do this for you. It installs and updates the tool, but it never distributes config content: a released version carries packaging metadata only, and its `preserve` list names *which* paths survive an update, never what is in them. This variable is the supported way, and it needs **no JARVIS change of any kind**.

Decide which boxes get it

Box	Set the variable?	Why
Networked worker that can reach the share	Yes	Central control, one file to edit
Air-gapped machine (e.g. the standalone A4000)	No — leave it unset	It cannot reach a share, and the paths inside a shared config are UNC paths that mean nothing there

This is deliberately opt-in per box. Every value in the config is a **path**, so a central copy is only meaningful to machines that can resolve those paths. A box with the variable unset behaves exactly as it did before 1.1.5 — nothing changes for it.

1. Put the config on a share

Pick a path every worker can read, e.g. `\\nas\gis\mc\shapefile_config.json`.

Grant the accounts the JARVIS agent runs as **read** access. Read is enough — the tool never writes this file, and it should not be writable by the boxes that consume it.

Seed it from a config you already trust:

```
copy "C:\JARVIS\tools\material_classification\.config\shapefile_config.json" "\\nas\gis\mc\sha
```

Then edit the shared copy so every path in it resolves **from the worker machines** — this is the step people get wrong. A value like `C:/ClassificationApp/sde/prod.sde` is a local path that will differ or not exist elsewhere. Make them UNC:

```
{
  "water_mask": " //nas/gis/mc/masks/aoi_water_mask.tif",
  "sde": {
    "enabled": true,
    "connection_file": " //nas/gis/mc/sde/prod_readonly.sde",
    "arcpy_python": "C:/Program Files/ArcGIS/Pro/bin/Python/envs/arcgispro-py3/python.exe",
    "road_width_attr": "WIDTH_M",
    "layers": { "buildings": "PROD.GIS.BUILDINGS", "roads": "PROD.GIS.ROADS" }
  }
}
```

Use forward slashes. `//nas/gis/...` is a valid absolute UNC path and needs no escaping. Backslashes in JSON must be doubled (`"\\nas\\gis\\ ... "`), which is the single most common way this file ends up malformed.

`arcpy_python` stays a **local** path — ArcGIS Pro is installed per machine, and it is normally the same path on every box.

Field-by-field meanings (SDE block, road-width tiers, `water_mask`) are in `README.txt` under "*Enterprise geodatabase (SDE / ArcGIS) setup*"; they are unchanged by this feature.

2. Point the boxes at it

Whole fleet — Group Policy (this is the "never visit every computer" part):

```
Computer Configuration
→ Preferences → Windows Settings → Environment
→ New → Environment Variable
  Action:    Update
  System variable (checked)
  Name:      MC_SHAPEFILE_CONFIG
  Value:     \\nas\gis\mc\shapefile_config.json
```

One box — elevated prompt:

```
setx MC_SHAPEFILE_CONFIG "\\nas\gis\mc\shapefile_config.json" /M
```

`/M` means **machine** scope, and it matters: the JARVIS agent may run as a service or under a different account than the person typing the command, and a user-scope variable would never reach it.

Do **not** put this in the tool's `start.bat` or inside a version folder — both are replaced wholesale on the next update, and the setting would silently vanish.

3. Restart the agent — this step is not optional

A running process never sees a new environment variable. The JARVIS agent captures its environment when it starts and hands that to every tool run. Until it restarts, the variable exists on the machine but not inside the agent, and **nothing will change**.

```
# stop the agent, then start it the normal way for this box, e.g.  
C:\jarvis\start-agent.bat
```

A reboot works too. If the agent runs as a service, restart the service.

4. Verify which config actually won

Every run now prints one line naming its source. Run the CLI on the box and look for it:

```
[shapefile_config] source: SHARED \\nas\gis\mc\shapefile_config.json (via MC_SHAPEFILE_CONFIG)
```

or, when the variable is unset:

```
[shapefile_config] source: local C:\JARVIS\tools\material_classification\1.1.5\shapefile_configi
```

If you still see **source: local** after setting the variable, the agent was not restarted (step 3) — that is nearly always the cause.

What happens when the share is unreachable

The run fails, loudly and immediately. It does not fall back to the local file:

```
ShapefileConfigError: MC_SHAPEFILE_CONFIG points at \\nas\gis\mc\shapefile_config.json,  
which could not be read: [Errno 2] No such file or directory: '\\\\nas\\gis\\mc\\shapefile_con  
Fix the path or the share, or unset MC_SHAPEFILE_CONFIG to use the local shapefile_config.json
```

(The doubled backslashes in the trailing quoted path are just Python echoing the path back — they are not a sign that your variable is set wrongly.)

This is intentional. You pointed the box at a central config; quietly classifying with a stale local one instead would produce plausible-looking output from the wrong attributes, and nothing in the results would reveal it. A stopped run is recoverable; silently wrong material rasters are not.

The same hard failure covers a file that is missing, unreadable, not valid JSON, or whose top level is not a JSON object. The message always names the path and the way out.

Note the asymmetry: a malformed **local** config is still tolerated and degrades to defaults, so an air-gapped box is never stopped mid-run by this change.

To roll back, remove the variable and restart the agent:

```
REG delete "HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment" /V MC_SHAPEFILE
```

The box returns to its local **.config** copy, which was never modified.

Gotchas

Symptom	Cause
Log still says <code>source: local</code>	Agent not restarted after setting the variable (step 3)
Works for you interactively, not for JARVIS tasks	Variable set in <code>user</code> scope; needs <code>/M</code> (machine)
Setting vanished after an update	It was put in <code>start.bat</code> or a version folder; use the machine environment
Runs fail with <code>ShapefileConfigError</code>	Share unreachable, path typo, or malformed JSON — the message names the path
Shared file loads but extraction finds nothing	Paths inside it are still <i>local</i> (<code>C:/ ...</code>) rather than UNC
Edited the installed <code>shapefile_config.json</code> , no effect	The shared file wins while the variable is set; edit the shared one

One trap that predates this feature

The per-box copy JARVIS preserves across updates lives at:

```
<tools_root>\material_classification\.config\shapefile_config.json
```

That copy is restored over every new install and **beats whatever the installer ships**. So putting new defaults into `shapefile_config.example.json` only affects boxes that have never installed the tool — an existing box keeps its `.config` copy. Editing that file is the right way to change one machine permanently without using the shared config.